Resultados de Tests — PlataformaVideojuegos

Alumno: DavidParada
Proyecto: PlataformaVideojuegos
Fecha: 2026-03-23

Resumen

Clase de Test	Total	✅ Pasados	❌ Fallidos
AppTest	1	1	0
UsuarioTest	35	33	2
JuegoTest	30	17	13
BibliotecaTest	18	17	1
CompraTest	23	10	13
ResenaTest	23	17	6
TOTAL	**130**	**95**	**35**

Causa raíz de fallos por clase

BibliotecaTest — 1 test fallido (17/18)

> **Motivo:** El test `consultaUltimaSesion_LanzaUnsupportedOperationException` espera que el alumno lance `UnsupportedOperationException`, pero el método devuelve normalmente sin lanzar excepción. El alumno implementó la funcionalidad en lugar de respetar el contrato que la declaraba como no soportada.

CompraTest — 13 tests fallidos (10/23)

> **Grupo A — 5 ERRORs (ValidationException inesperada en `procesarPago` / `consultarCompra`):** Los tests que llaman a `procesarPago` o `consultarCompra` tras una compra válida reciben una `ValidationException` inesperada. El `PurchaseController` lanza una validación internamente al procesar el pago o consultar la compra, impidiendo que estos flujos funcionen correctamente.
>
> **Grupo B — 2 FAILUREs (`IllegalArgumentException` en lugar de `ValidationException`):** Cuando el usuario o el juego no existen, el `PurchaseController` lanza `IllegalArgumentException` (vía `orElseThrow`) en vez de `ValidationException`. El `CompraAdapter` no captura este tipo de excepción ni la convierte.
>
> **Grupo C — 5 FAILUREs (validaciones ausentes):** El alumno no valida: método de pago nulo, descuento negativo, descuento > 100, juego no disponible ni precio negativo en el formulario de compra.
>
> **Grupo D — 1 FAILURE (mapeo erróneo):** `realizarCompra_DescuentoCero_PorDefecto` — el campo `descuentoAplicado` del DTO devuelve el precio base (`29.989…`) en lugar de `0.0`. El `CompraDTOMapper` asigna el campo precio al campo descuento.

ResenaTest — 6 tests fallidos (17/23)

> **2 ERRORs (`otroUsuario` con contraseña corta):** Los tests `eliminarResena_UsuarioNoEsDuenio` y `ocultarResena_UsuarioNoEsDuenio` crean un segundo usuario dentro del propio test con contraseña `"12345678Aa@"` (11 chars), que sigue siendo rechazada por la validación del alumno (`PASS_L = 20`). Al fallar esa creación, el test arroja un error inesperado antes de llegar al `assertThrows`.
>
> **4 FAILUREs (lógica incompleta):** El alumno no valida reseñas duplicadas del mismo usuario + juego; `eliminarResena` no elimina la reseña del repositorio; `ocultarResena` no filtra la reseña del listado público; `listarResenasPorUsuario` no lanza `ValidationException` cuando el usuario no

> existe.

JuegoTest — 13 tests fallidos

> **Motivo A (11 tests):** Al crear un juego con precio `29.99` (double), el mapper lo convierte a `float`, lo que introduce imprecisión de coma flotante (`(float)29.99 → 29.989999...`). El alumno evalúa la precisión con `BigDecimal.valueOf((float) precio)`, cuya escala supera 2 decimales → error `FORMATO_INVALIDO` → `ValidationException`.
>
> **Motivo B (1 test — `crearJuego_FormularioValido_IdiomasNulos_Permitido`):** El mapper convierte `null` en idiomas a `List.of()` (lista vacía). El alumno llama a `availabeLanguages.getFirst()` sobre la lista vacía → `NoSuchElementException` → capturada por el adapter y relanzada como `ValidationException`.
>
> **Cascada:** Los tests que dependen de que exista un juego previo (`buscarJuegos`, `listarCatalogo`, `aplicarDescuento`, `cambiarEstado`) fallan a su vez porque la creación del juego válido fracasa.

UsuarioTest — 2 tests fallidos

> **Motivo:** El alumno lanza `IllegalArgumentException("Usuario no encontrado")` en `UserController.showUserProfile()` cuando el id o nombre de usuario es inválido. El `UsuarioAdapter.consultarPerfil()` solo captura `org.example.Exeptions.ValidationException`, por lo que la `IllegalArgumentException` se propaga sin capturar. Los tests esperaban que se devolviera `null`.

AppTest

Test	Resultado
`shouldAnswerWithTrue`	✅ PASS

UsuarioTest (33/35)

Test	Resultado	Motivo del fallo
`crearUsuarioDTO_FormularioValido_RetornaUsuarioDTO`	✅ PASS	
`crearUsuarioDTO_FormularioValido_AvatarNulo_Permitido`	✅ PASS	
`crearUsuarioDTO_FormularioValido_FechaRegistroGeneradaAutomaticamente`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreObligatorio`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreMenor3Caracteres`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreMayor20Caracteres`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreConCaracteresInvalidos`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreEmpiezaConNumero`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreNoUnico`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_EmailObligatorio`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_EmailFormatoInvalido`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_EmailNoUnico`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_ContrasenaObligatoria`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_ContrasenaMenor8Caracteres`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_ContrasenaSinMayuscula`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_ContrasenaSinMinuscula`	✅ PASS	

Test	Resultado	Motivo del fallo
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_ContrasenaSinNumero`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreRealObligatorio`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreRealMenor2Caracteres`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_NombreRealMayor50Caracteres`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_PaisObligatorio`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_PaisNoValido`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_FechaNacimientoObligatoria`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_FechaNacimientoFutura`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_FechaNacimientoMenor13Anos`	✅ PASS	
`crearUsuarioDTO_FormularioInvalido_LanzaValidationException_AvatarMayor100Caracteres`	✅ PASS	
`consultarPerfil_IdValido_RetornaUsuarioDTO`	✅ PASS	
`consultarPerfil_NombreUsuarioValido_RetornaUsuarioDTO`	✅ PASS	
`consultarSaldo_IdValido_RetornaSaldo`	✅ PASS	
`consultarSaldo_IdInvalido_LanzaValidationException`	✅ PASS	
`aniadirSaldo_IdValido_CantidadValida_RetornaUsuarioDTOConSaldoActualizado`	✅ PASS	
`aniadirSaldo_CantidadNoValida_LanzaValidationException`	✅ PASS	
`aniadirSaldo_IdInvalido_LanzaValidationException`	✅ PASS	
`consultarPerfil_IdInvalido_RetornaNull`	❌ FAIL	`UserController.showUserProfile()` lanza `IllegalArgumentException` cuando el usuario no existe; el adapter no captura este tipo de excepción y devuelve null solo para `ValidationException`. La excepción se propaga sin capturar.
`consultarPerfil_NombreUsuarioInvalido_RetornaNull`	❌ FAIL	Mismo motivo: `IllegalArgumentException` no capturada en `UsuarioAdapter.consultarPerfil(String)`

JuegoTest (17/30)

Test	Resultado	Motivo del fallo
`crearJuego_FormularioValido_RetornaJuegoDTO`	❌ FAIL	Precio `29.99` (double→float) introduce imprecisión de coma flotante; `BigDecimal.valueOf` detecta más de 2 decimales → `FORMATO_INVALIDO`.
`crearJuego_FormularioInvalido_LanzaValidationException_TituloObligatorio`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_TituloMayor100Caracteres`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_TituloNoUnico`	❌ FAIL	Dependía de crear primero un juego válido con precio `29.99`; la creación falla por el motivo anterior.
`crearJuego_FormularioValido_DescripcionNula_Permitida`	❌ FAIL	Precio `19.99` (double→float) → misma imprecisión de coma flotante.

Test	Resultado	Motivo del fallo
`crearJuego_FormularioInvalido_LanzaValidationException_DescripcionMayor2000Caracteres`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_DesarrolladorObligatorio`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_DesarrolladorMenor2Caracteres`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_DesarrolladorMayor100Caracteres`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_FechaLanzamientoObligatoria`	✅ PASS	
`crearJuego_FormularioValido_FechaLanzamientoFutura_Permitida`	❌ FAIL	Precio `29.99` (double→float) → imprecisión de coma flotante.
`crearJuego_FormularioValido_PrecioBaseCero_Permitido`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_PrecioBaseNegativo`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_PrecioBaseSuperaMaximo`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_DescuentoNegativo`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_DescuentoMayor100`	✅ PASS	
`crearJuego_FormularioInvalido_LanzaValidationException_ClasificacionEdadObligatoria`	✅ PASS	
`crearJuego_FormularioValido_IdiomasNulos_Permitido`	❌ FAIL	El mapper convierte `null` a `List.of()` (vacía); el alumno llama `availabeLanguages.getFirst()` sobre lista vacía → `NoSuchElementException` capturada como `ValidationException`.
`crearJuego_FormularioInvalido_LanzaValidationException_IdiomasArrayVacio`	✅ PASS	
`crearJuego_FormularioValido_EstadoDisponible_PorDefecto`	❌ FAIL	Precio `29.99` (double→float) → imprecisión de coma flotante.
`crearJuego_FormularioValido_DescuentoCero_PorDefecto`	❌ FAIL	Precio `29.99` (double→float) → imprecisión de coma flotante.
`listarCatalogo_SinJuegos_RetornaListaVacia`	✅ PASS	
`listarCatalogo_ConJuegos_RetornaJuegosDisponibles`	❌ FAIL	La creación previa del juego (precio `29.99`) falla por imprecisión en float; no existe ningún juego en el catálogo.
`buscarJuegos_TextoCoincidente_RetornaResultados`	❌ FAIL	Ídem: la creación previa del juego falla por conversión double→float.
`buscarJuegos_TextoSinCoincidencia_RetornaListaVacia`	❌ FAIL	Ídem: la creación previa del juego falla por conversión double→float.
`aplicarDescuento_IdValido_DescuentoValido_RetornaJuegoActualizado`	❌ FAIL	Ídem: la creación previa del juego falla; no existe juego sobre el que aplicar el descuento.
`aplicarDescuento_IdInvalido_LanzaValidationException`	✅ PASS	
`aplicarDescuento_DescuentoFueraDeRango_LanzaValidationException`	❌ FAIL	Ídem: la creación previa del juego falla; el test no puede probar el descuento fuera de rango.
`cambiarEstado_IdValido_EstadoValido_RetornaJuegoConNuevoEstado`	❌ FAIL	Ídem: la creación previa del juego falla.
`cambiarEstado_IdInvalido_LanzaValidationException`	✅ PASS	

BibliotecaTest (17/18)

Test	Resultado	Motivo del fallo
`obtenerJuegosUsuario_UsuarioValido_RetornaListaVacia`	✅ PASS	
`obtenerJuegosUsuario_UsuarioInvalido_LanzaValidationException`	✅ PASS	
`agregarJuegoBiblioteca_UsuarioYJuegoValidos_RetornaTrue`	✅ PASS	
`agregarJuegoBiblioteca_VerificaEntradaEnBiblioteca`	✅ PASS	
`agregarJuegoBiblioteca_TiempoJuegoInicialCero`	✅ PASS	
`agregarJuegoBiblioteca_EstadoInstalacionPorDefectoNoInstalado`	✅ PASS	
`agregarJuegoBiblioteca_JuegoDuplicado_LanzaValidationException`	✅ PASS	
`agregarJuegoBiblioteca_JuegoInexistente_LanzaValidationException`	✅ PASS	
`agregarJuegoBiblioteca_UsuarioInexistente_LanzaValidationException`	✅ PASS	
`agregarJuegoBiblioteca_OtroUsuarioMismoJuego_Permitido`	✅ PASS	
`eliminarJuegoBiblioteca_EntradaExistente_EliminaCorrectamente`	✅ PASS	
`eliminarJuegoBiblioteca_EntradaInexistente_LanzaValidationException`	✅ PASS	
`eliminarJuegoBiblioteca_UsuarioInexistente_LanzaValidationException`	✅ PASS	
`actualizarTiempoJuego_EntradaValida_TiempoActualizado`	✅ PASS	
`actualizarTiempoJuego_TiempoNegativo_LanzaValidationException`	✅ PASS	
`actualizarTiempoJuego_JuegoNoEnBiblioteca_LanzaValidationException`	✅ PASS	
`actualizarTiempoJuego_UsuarioInexistente_LanzaValidationException`	✅ PASS	
`consultaUltimaSesion_LanzaUnsupportedOperationException`	❌ FAIL	El test espera que se lance `UnsupportedOperationException` , pero el método devuelve sin lanzar excepción. El alumno implementó la funcionalidad en lugar de respetar el contrato no soportado.

CompraTest (10/23)

Test	Resultado	Motivo del fallo
`realizarCompra_FormularioValido_RetornaCompraDTO`	✅ PASS	
`realizarCompra_EstadoPorDefectoPendiente`	✅ PASS	
`realizarCompra_FechaCompraGeneradaAutomaticamente`	✅ PASS	
`realizarCompra_PrecioSinDescuentoCero_permitido`	✅ PASS	
`realizarCompra_JuegoEnPreventa_Permitido`	✅ PASS	
`realizarCompra_JuegoEnAccesoAnticipado_Permitido`	✅ PASS	
`consultarCompra_IdInvalido_LanzaValidationException`	✅ PASS	
`procesarPago_IdInvalido_LanzaValidationException`	✅ PASS	
`solicitarReembolso_CompraPendiente_LanzaValidationException`	✅ PASS	
`solicitarReembolso_IdInvalido_LanzaValidationException`	✅ PASS	

Test	Resultado	Motivo del fallo
`realizarCompra_UsuarioInexistente_LanzaValidationException`	❌ FAIL	El `PurchaseController` lanza `IllegalArgumentException` (vía `orElseThrow`) cuando el usuario no existe. El `CompraAdapter` no la captura ni la convierte a `ValidationException`.
`realizarCompra_JuegoInexistente_LanzaValidationException`	❌ FAIL	Mismo motivo: `IllegalArgumentException` no capturada en el adapter cuando el juego no existe.
`realizarCompra_JuegoNoDisponible_LanzaValidationException`	❌ FAIL	El alumno no valida que el juego deba estar en un estado que permita la compra; acepta el juego en estado `NO_DISPONIBLE`.
`realizarCompra_MetodoPagoNulo_LanzaValidationException`	❌ FAIL	El alumno no valida que el método de pago sea obligatorio; acepta `null` sin lanzar excepción.
`realizarCompra_PrecioSinDescuentoNegativo_LanzaValidationException`	❌ FAIL	El alumno no valida que el precio de compra no puede ser negativo.
`realizarCompra_DescuentoNegativo_LanzaValidationException`	❌ FAIL	El alumno no valida que el descuento aplicado en la compra no puede ser negativo.
`realizarCompra_DescuentoMayor100_LanzaValidationException`	❌ FAIL	El alumno no valida que el descuento aplicado en la compra no puede superar 100.
`realizarCompra_DescuentoCero_PorDefecto`	❌ FAIL	El campo `descuentoAplicado` retorna el precio base (`29.989…`) en lugar de `0.0`. El `CompraDTOMapper` asigna erróneamente el precio al campo de descuento.
`procesarPago_CompraEnEstadoPendiente_RetornaCompraCompletada`	❌ FAIL	`procesarPago` lanza `ValidationException` inesperadamente tras una compra válida. El `PurchaseController` falla con su propia validación interna al intentar procesar el pago.
`procesarPago_CompraYaCompletada_LanzaValidationException`	❌ FAIL	La primera llamada a `procesarPago` dentro del test ya lanza `ValidationException` (mismo bug anterior); el `assertThrows` nunca se alcanza.
`consultarCompra_IdValidoUsuarioCorrecto_RetornaCompraDTO`	❌ FAIL	`consultarCompra` lanza `ValidationException` inesperadamente. El `PurchaseController` falla internamente al intentar recuperar la compra por id y usuario.
`consultarCompra_UsuarioNoEsDuenio_LanzaValidationException`	❌ FAIL	`consultarCompra` lanza `ValidationException` inesperada antes de verificar la condición de dueño (mismo bug de `consultarCompra`).
`solicitarReembolso_CompraCompletada_RetornaCompraReembolsada`	❌ FAIL	La llamada previa a `procesarPago` falla con `ValidationException` (mismo bug); `solicitarReembolso` nunca se alcanza.

ResenaTest (17/23)

Test	Resultado	Motivo del fallo
`crearResena_FormularioValido_RetornaResenaDTO`	✅ PASS	
`crearResena_TextoVacio_LanzaValidationException`	✅ PASS	
`crearResena_TextoMenor50Caracteres_LanzaValidationException`	✅ PASS	
`crearResena_TextoMayor8000Caracteres_LanzaValidationException`	✅ PASS	
`crearResena_UsuarioInexistente_LanzaValidationException`	✅ PASS	

Test	Resultado	Motivo del fallo
`crearResena_JuegoInexistente_LanzaValidationException`	✅ PASS	
`crearResena_UsuarioSinJuegoEnBiblioteca_LanzaValidationException`	✅ PASS	
`crearResena_HorasJugadasObtenidaDeBiblioteca`	✅ PASS	
`crearResena_EstadoPorDefectoPublicada`	✅ PASS	
`crearResena_FechaPublicacionGeneradaAutomaticamente`	✅ PASS	
`listarResenasJuego_JuegoSinResenas_RetornaListaVacia`	✅ PASS	
`listarResenasJuego_JuegoCon1Resena_RetornaListaConUnaResena`	✅ PASS	
`listarResenasJuego_JuegoInexistente_LanzaValidationException`	✅ PASS	
`listarResenasPorUsuario_UsuarioSinResenas_RetornaListaVacia`	✅ PASS	
`listarResenasPorUsuario_UsuarioConResenas_RetornaLista`	✅ PASS	
`eliminarResena_IdInvalido_LanzaValidationException`	✅ PASS	
`ocultarResena_IdInvalido_LanzaValidationException`	✅ PASS	
`crearResena_ResenaDuplicada_LanzaValidationException`	❌ FAIL	El alumno no valida que un mismo usuario no puede tener más de una reseña por juego; la segunda creación no lanza excepción.
`eliminarResena_ResenaPropiaExistente_EliminaCorrectamente`	❌ FAIL	`eliminarResena` no elimina la reseña del repositorio; el listado posterior del juego sigue incluyéndola.
`eliminarResena_UsuarioNoEsDuenio_LanzaValidationException`	❌ FAIL	El test crea `otroUsuario` con contraseña `"12345678Aa@"` (11 chars), que sigue siendo rechazada por la validación del alumno (`PASS_L = 20`). La `ValidationException` se lanza en la creación del usuario, no en `eliminarResena` .
`ocultarResena_ResenaPropiaPublicada_QuedaOculta`	❌ FAIL	`ocultarResena` no cambia el estado de la reseña a oculto de forma que afecte al listado público; la reseña sigue apareciendo en `listarResenasJuego` .
`ocultarResena_UsuarioNoEsDuenio_LanzaValidationException`	❌ FAIL	Mismo problema que `eliminarResena_UsuarioNoEsDuenio` : la creación de `otroUsuario` con contraseña corta lanza la excepción antes de llegar a `ocultarResena` .
`listarResenasPorUsuario_UsuarioInexistente_LanzaValidationException`	❌ FAIL	El alumno no lanza `ValidationException` cuando el usuario no existe; el método devuelve sin excepción.